

Networks and Causal Inference Short Course: Part 1

Claire Pearsall, Ashley Buchanan

2026-08-03

Using Data from the Transmission Reduction Intervention Project in Athens, Greece

1. Step 1: Download the data and load it into R Studio

This is a simulated toy dataset made for this course. It is based on data collected from the Transmission Reduction Intervention Project in Athens Greece. Public data here: <https://www.icpsr.umich.edu/web/NAHDAP/studies/39059/versions/V2/staff>

Read more: <https://pubmed.ncbi.nlm.nih.gov/27917890/>

```
#install packages (if needed)
#install.packages(c("readxl", "dplyr", "ggplot2", "geepack", "tidyverse"))
#load packages
library(readxl)
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.2
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(knitr)
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
library(geepack)
```

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.5.2
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v forcats   1.0.1      v stringr   1.6.0
```

```
## v lubridate 1.9.4      v tibble   3.3.0
```

```
## v purrr     1.2.1      v tidyr    1.3.1
```

```
## v readr     2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(cobalt)
```

```
## Warning: package 'cobalt' was built under R version 4.5.2
```

```
## cobalt (Version 4.6.2, Build Date: 2026-01-29)
```

```
library(purrr)
```

```
library(scales)
```

```
##
```

```
## Attaching package: 'scales'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
##     discard
```

```
##
```

```
## The following object is masked from 'package:readr':
```

```
##
```

```
##     col_factor
```

```
# -----
```

```
# 1. Read data into R
```

```
# -----
```

```
#will need to test on final folder structure shared - we can post to GitHub
```

```
nodes <- read.csv("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/main/data/nodes.csv")
```

```
edges <- read.csv("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/main/data/edges.csv")
```

```
head(nodes)
```

```
##   id num_nn component posneg date share education employment age avg_posneg
## 1  1     4         1      1    1     1         0           1  39  1.0000000
## 2  2     3         1      1    0     0         1           0  25  0.6666667
## 3  3     1         1      1    0     1         0           1  40  1.0000000
## 4  4     3         1      0    1     1         1           0  37  0.6666667
## 5  5     4         1      1    0     1         1           0  28  0.5000000
## 6  6     2         1      1    1     0         0           1  17  1.0000000
##   avg_date avg_share avg_education avg_employment avg_age   raneff treatment
## 1 0.5000000 0.5000000   0.7500000   0.5000000    27.75 -0.1141261         0
## 2 1.0000000 1.0000000   0.6666667   0.6666667    35.67 -0.1141261         0
## 3 0.0000000 1.0000000   0.0000000   1.0000000    31.00 -0.1141261         1
## 4 0.3333333 0.3333333   0.6666667   0.3333333    29.00 -0.1141261         0
## 5 0.7500000 0.5000000   0.5000000   0.2500000    36.75 -0.1141261         0
## 6 1.0000000 0.5000000   0.5000000   0.5000000    38.00 -0.1141261         1
##   num_tnn num_utnn outcome
## 1      0      4       1
## 2      0      3       0
## 3      0      1       1
## 4      0      3       0
## 5      1      3       1
## 6      1      1       0
```

```
head(edges)
```

```
##   from to
## 1    1  2
## 2    2  4
```

```
## 3    5  8
## 4    1  9
## 5    3  9
## 6    1 10
```

Generate descriptive statistical analyses and histograms

```
# -----
# 2. Descriptive Analysis
# -----

# Basic summary statistics
summary(nodes)

##          id          num_nn      component      posneg
## Min.   : 1.00   Min.   :1.00   Min.   : 1.000   Min.   :0.0000
## 1st Qu.: 54.75   1st Qu.:2.00   1st Qu.: 3.000   1st Qu.:0.0000
## Median :108.50   Median :3.00   Median : 5.000   Median :1.0000
## Mean   :108.50   Mean   :3.25   Mean   : 6.333   Mean   :0.5278
## 3rd Qu.:162.25   3rd Qu.:4.00   3rd Qu.: 9.000   3rd Qu.:1.0000
## Max.   :216.00   Max.   :8.00   Max.   :20.000   Max.   :1.0000
##          date          share      education      employment
## Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.000
## 1st Qu.:0.0000   1st Qu.:1.0000   1st Qu.:0.0000   1st Qu.:0.000
## Median :1.0000   Median :1.0000   Median :0.0000   Median :0.000
## Mean   :0.5185   Mean   :0.7685   Mean   :0.4167   Mean   :0.463
## 3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:1.000
## Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   Max.   :1.000
##          age      avg_posneg      avg_date      avg_share
## Min.   :14.00   Min.   :0.0000   Min.   :0.0000   Min.   :0.000
## 1st Qu.:29.00   1st Qu.:0.3333   1st Qu.:0.3333   1st Qu.:0.575
## Median :35.00   Median :0.5000   Median :0.5000   Median :0.800
## Mean   :34.86   Mean   :0.5328   Mean   :0.5319   Mean   :0.767
## 3rd Qu.:40.00   3rd Qu.:0.7500   3rd Qu.:0.7500   3rd Qu.:1.000
## Max.   :58.00   Max.   :1.0000   Max.   :1.0000   Max.   :1.000
## avg_education  avg_employment      avg_age      raneff
## Min.   :0.0000   Min.   :0.0000   Min.   :19.00   Min.   : -1.096824
## 1st Qu.:0.0000   1st Qu.:0.2500   1st Qu.:31.25   1st Qu.: -0.194025
## Median :0.4143   Median :0.5000   Median :34.71   Median : -0.083398
## Mean   :0.4223   Mean   :0.4508   Mean   :34.54   Mean   : -0.004333
## 3rd Qu.:0.6667   3rd Qu.:0.6667   3rd Qu.:37.50   3rd Qu.: 0.212757
## Max.   :1.0000   Max.   :1.0000   Max.   :53.00   Max.   : 0.870289
##          treatment      num_tnn      num_utnn      outcome
## Min.   :0.0000   Min.   :0.0000   Min.   :0.000   Min.   :0.0000
## 1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:2.000   1st Qu.:0.0000
## Median :0.0000   Median :0.0000   Median :3.000   Median :0.0000
## Mean   :0.1296   Mean   :0.3565   Mean   :2.894   Mean   :0.4861
## 3rd Qu.:0.0000   3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:1.0000
## Max.   :1.0000   Max.   :3.0000   Max.   :8.000   Max.   :1.0000

# Labels for table
# Variable labels
var_labels <- c(
  posneg      = "HIV status",
  share       = "Shared needles",
```

```

education = "Education",
employment = "Employment status",
treatment = "Treatment assignment",
outcome = "Outcome",
num_nn = "Number of network neighbors",
num_tnn = "Number of treated neighbors",
num_utnn = "Number of untreated neighbors"
)

# Level labels
level_labels <- list(
  posneg = c(
    "0" = "HIV negative",
    "1" = "HIV positive"
  ),
  share = c(
    "0" = "No",
    "1" = "Yes"
  ),
  education = c(
    "0" = "Less than high school",
    "1" = "High school or higher"
  ),
  employment = c(
    "0" = "Unemployed",
    "1" = "Employed"
  ),
  treatment = c(
    "0" = "Control",
    "1" = "Treatment"
  ),
  outcome = c(
    "0" = "Negative outcome",
    "1" = "Positive outcome"
  )
)

make_table1 <- function(data, cat_vars, num_vars,
                        var_labels = NULL,
                        level_labels = NULL) {

  # Categorical variables
  cat_table <- purrr::map_dfr(cat_vars, function(var) {

    # Header row
    header <- tibble(
      Level = if (!is.null(var_labels)) var_labels[[var]] else var,
      `Count (%)` = ""
    )

    # Data rows
    rows <- data %>%
      count(.data[[var]]) %>%

```

```

    mutate(
      Level = as.character(.data[[var]])
    )

    if (!is.null(level_labels) && var %in% names(level_labels)) {
      rows$Level <- dplyr::recode(rows$Level, !!!level_labels[[var]])
    }

    rows <- rows %>%
      transmute(
        Level = paste0(" ", Level),
        `Count (%)` = paste0(
          n, " (", scales::percent(n / sum(n), accuracy = 0.1), ")"
        )
      )

    bind_rows(header, rows)
  })

  # Numeric variables
  num_table <- purrr::map_dfr(num_vars, function(var) {

    data %>%
      summarise(
        mean = mean(.data[[var]], na.rm = TRUE),
        sd = sd(.data[[var]], na.rm = TRUE)
      ) %>%
      transmute(
        Level = if (!is.null(var_labels)) var_labels[[var]] else var,
        `Count (%)` = sprintf("%.2f (%.2f)", mean, sd)
      )
  })

  bind_rows(cat_table, num_table)
}

table1 <- make_table1(
  data = nodes,
  cat_vars = c(
    "posneg",
    "share",
    "education",
    "employment",
    "treatment",
    "outcome"
  ),
  num_vars = c(
    "num_nn",
    "num_tnn",
    "num_utnn"
  ),
  var_labels = var_labels,
  level_labels = level_labels

```

```
)

table1

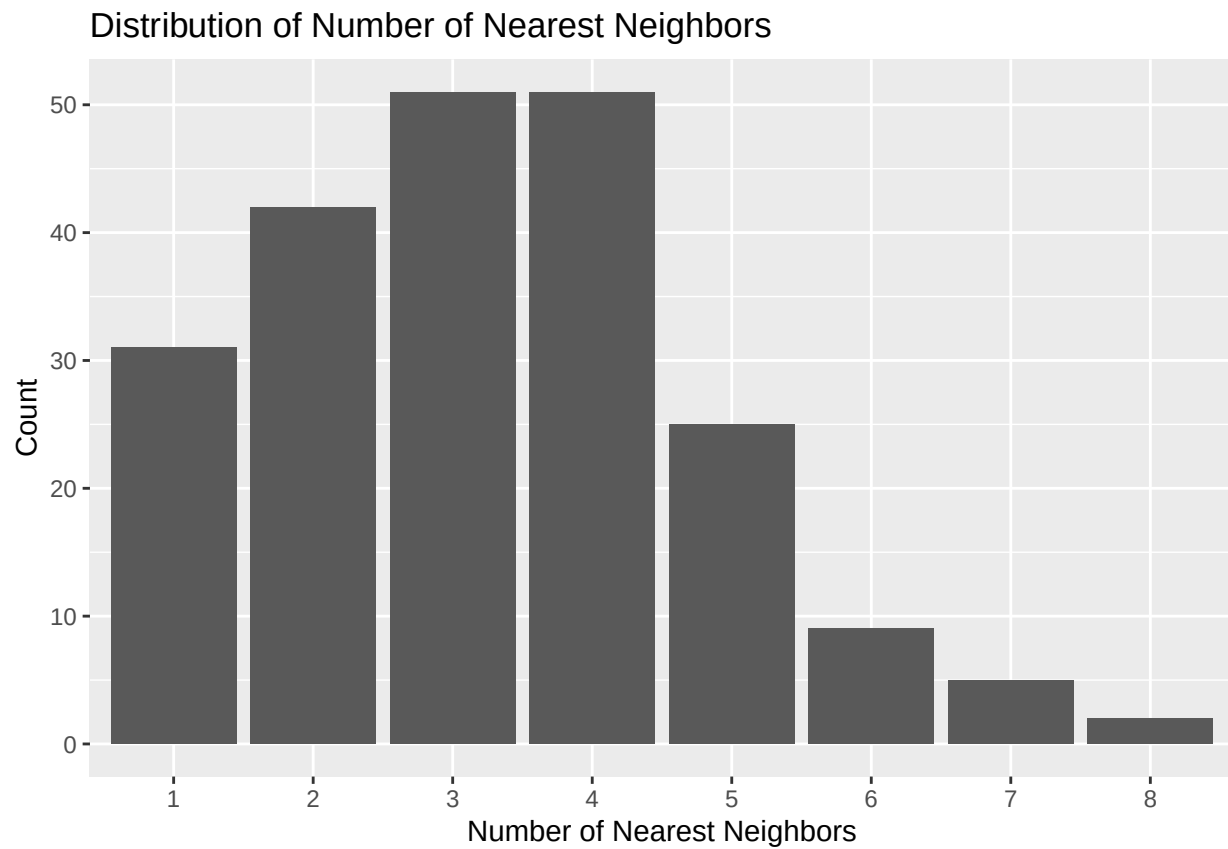
## # A tibble: 21 x 2
##   Level                                `Count (%)`
##   <chr>                                <chr>
## 1 "HIV status"                        ""
## 2 " HIV negative"                    "102 (47.2%)"
## 3 " HIV positive"                    "114 (52.8%)"
## 4 "Shared needles"                  ""
## 5 " No"                              "50 (23.1%)"
## 6 " Yes"                             "166 (76.9%)"
## 7 "Education"                        ""
## 8 " Less than high school"           "126 (58.3%)"
## 9 " High school or higher"           "90 (41.7%)"
## 10 "Employment status"               ""
## # i 11 more rows
```

```
# Means of key numeric variables
nodes %>%
  summarise(
    mean_nn = mean(num_nn, na.rm = TRUE),
    mean_tnn = mean(num_tnn, na.rm = TRUE),
    mean_utnn = mean(num_utnn, na.rm = TRUE)
  )
```

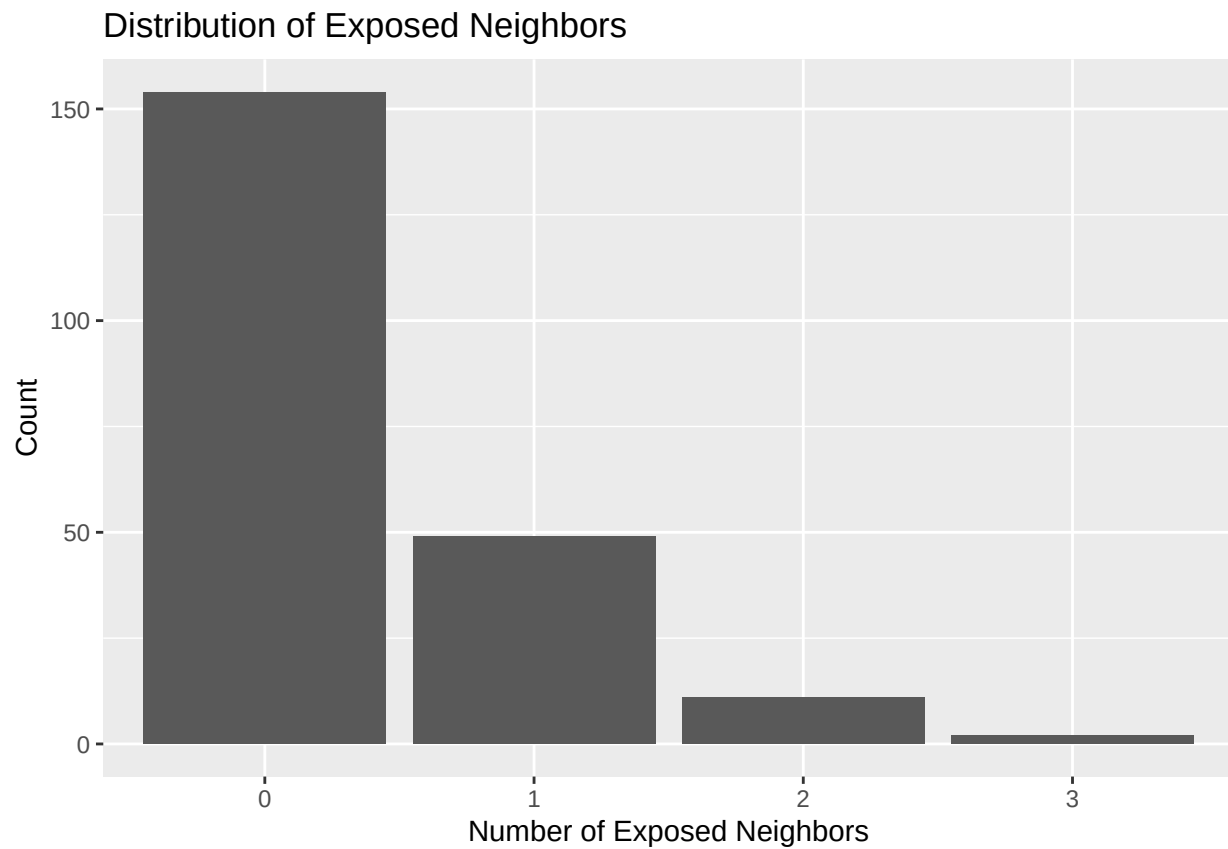
```
##   mean_nn mean_tnn mean_utnn
## 1    3.25 0.3564815 2.893519
```

```
# Plots of network-related variables

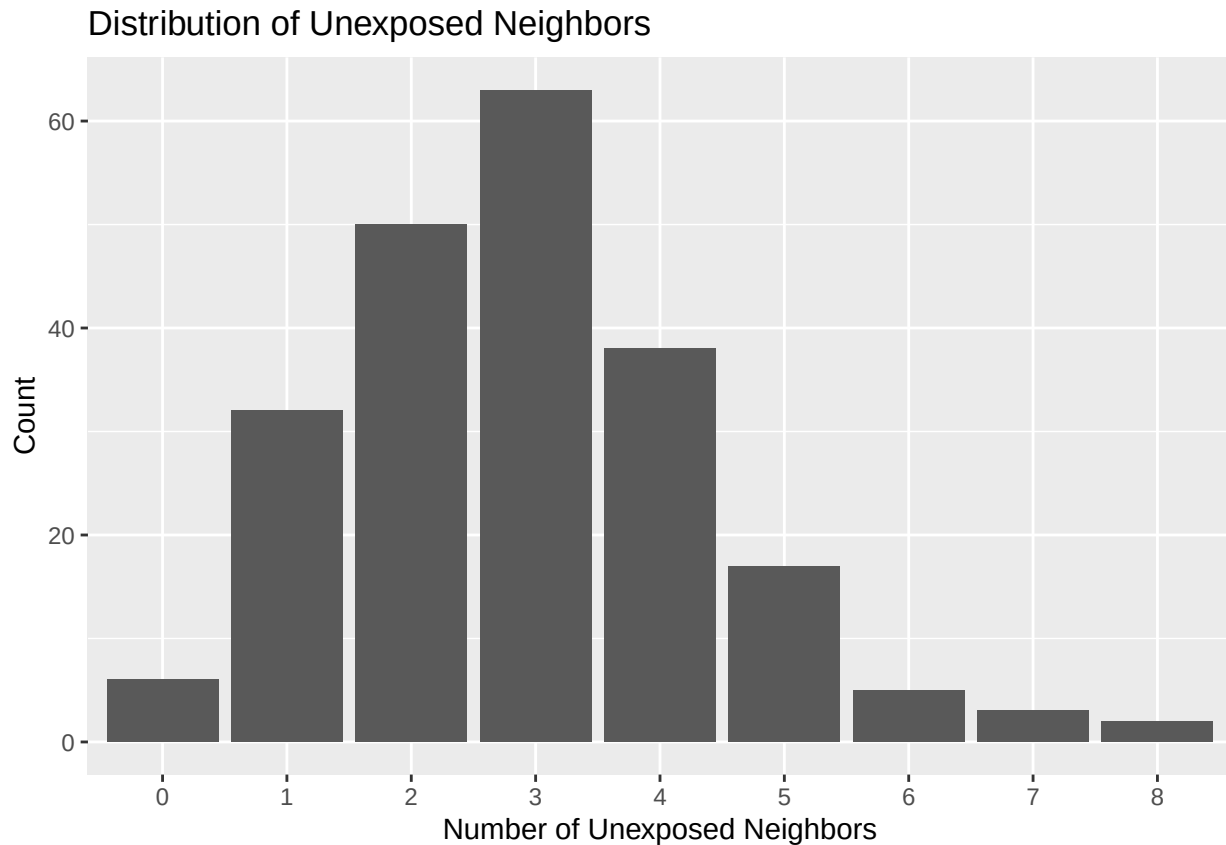
ggplot(nodes, aes(x = factor(num_nn))) +
  geom_bar() +
  scale_x_discrete(breaks = 0:9, labels = 0:9) +
  labs(title = "Distribution of Number of Nearest Neighbors",
       x = "Number of Nearest Neighbors",
       y = "Count")
```



```
ggplot(nodes, aes(x = factor(num_tnn))) +  
  geom_bar() +  
  scale_x_discrete(breaks = 0:4, labels = 0:4) +  
  labs(title = "Distribution of Exposed Neighbors",  
        x = "Number of Exposed Neighbors",  
        y = "Count")
```



```
ggplot(nodes, aes(x = factor(num_utnn))) +  
  geom_bar() +  
  scale_x_discrete(breaks = 0:8, labels = 0:8) +  
  labs(title = "Distribution of Unexposed Neighbors",  
        x = "Number of Unexposed Neighbors",  
        y = "Count")
```

Analysis of the association between community alerts and any risk behavior outcome, unadjusted and adjusted for baseline covariates.

```
# -----  
# 3. Assess association  
# -----  
  
#Conditional RD, RR - condition on covariates in outcome model (not marginal)  
  
#Unadjusted models  
#Risk Difference  
  
model_rd_unadj <- glm(  
  outcome ~ treatment,  
  data = nodes,  
  family = binomial(link = "identity")  
)  
  
#Risk Ratio  
model_rr_unadj <- glm(  
  outcome ~ treatment,  
  data = nodes,  
  family = binomial(link = "log")  
)  
  
#adjusted models
```

```

#Risk Difference
# Binomial Log model did not converge - fit with GEE log-Poisson to get a robust
# estimator of the variance

# rescale age (per 10 year increase)
nodes$age10 <- nodes$age/10

# model_rr <- glm(
#   outcome ~ treatment + age10 + posneg + share + education + employment,
#   data = nodes,
#   family = binomial(link = "log")
# )

gee_rr <- geeglm(
  outcome ~ treatment + age10 + posneg + date + share + education + employment,
  data = nodes,
  id = id,
  family = poisson(link = "log"),
  corstr = "independence"
)

#Risk Ratio
# Binomial Identity model did not converge - fit with GEE Normal-identity to get a robust
# estimator of the variance
# model_rd <- glm(
#   outcome ~ treatment + posneg + share + education + employment,
#   data = nodes,
#   family = binomial(link = "identity")
# )

gee_rd <- geeglm(
  outcome ~ treatment + age10 + posneg + date + share + education + employment,
  data = nodes,
  id = id,
  family = gaussian(link = "identity"),
  corstr = "independence"
)

#Format output into a table
# Associational RR and RD

# Unadjusted RR
b_rr_unadj <- coef(model_rr_unadj)["treatment"]
se_rr_unadj <- sqrt(vcov(model_rr_unadj)["treatment", "treatment"])
rr_unadj <- exp(b_rr_unadj)
rr_unadj_ci <- exp(b_rr_unadj + c(-1, 1) * 1.96 * se_rr_unadj)

# Unadjusted RD
b_rd_unadj <- coef(model_rd_unadj)["treatment"]
se_rd_unadj <- sqrt(vcov(model_rd_unadj)["treatment", "treatment"])
rd_unadj <- b_rd_unadj
rd_unadj_ci <- b_rd_unadj + c(-1, 1) * 1.96 * se_rd_unadj

```

```

# Adjusted RR
b_rr_adj <- coef(gee_rr)["treatment"]
se_rr_adj <- sqrt(vcov(gee_rr)["treatment", "treatment"])
rr_adj <- exp(b_rr_adj)
rr_adj_ci <- exp(b_rr_adj + c(-1, 1) * 1.96 * se_rr_adj)

# Adjusted RD
b_rd_adj <- coef(gee_rd)["treatment"]
se_rd_adj <- sqrt(vcov(gee_rd)["treatment", "treatment"])
rd_adj <- b_rd_adj
rd_adj_ci <- b_rd_adj + c(-1, 1) * 1.96 * se_rd_adj

results <- data.frame(
  model = c("Unadjusted", "Unadjusted", "Adjusted", "Adjusted"),
  measure = c("Risk ratio", "Risk difference", "Risk ratio", "Risk difference"),
  estimate = c(rr_unadj, rd_unadj, rr_adj, rd_adj),
  lower_95_ci = c(rr_unadj_ci[1], rd_unadj_ci[1], rr_adj_ci[1], rd_adj_ci[1]),
  upper_95_ci = c(rr_unadj_ci[2], rd_unadj_ci[2], rr_adj_ci[2], rd_adj_ci[2])
)

results

##           model           measure      estimate lower_95_ci upper_95_ci
## 1 Unadjusted      Risk ratio    1.032967033    0.6933062    1.5390327
## 2 Unadjusted Risk difference    0.015957447   -0.1825452    0.2144601
## 3   Adjusted      Risk ratio    1.024567106    0.7286820    1.4405979
## 4   Adjusted Risk difference   -0.001150168   -0.1754597    0.1731593

#Round to 3 decimal places
results$estimate <- round(results$estimate, 3)
results$lower_95_ci <- round(results$lower_95_ci, 3)
results$upper_95_ci <- round(results$upper_95_ci, 3)

results

##           model           measure estimate lower_95_ci upper_95_ci
## 1 Unadjusted      Risk ratio      1.033         0.693         1.539
## 2 Unadjusted Risk difference      0.016        -0.183         0.214
## 3   Adjusted      Risk ratio      1.025         0.729         1.441
## 4   Adjusted Risk difference     -0.001        -0.175         0.173

```

Check the positivity assumption for random violations, create tables of the exposure to the community alerts by each covariate. Based on the exposure, there are no structural violations of positivity (exposure possible for all combinations of individual and group level covariates).

```

# -----
# 5. Assess random violations of positivity
# -----
# Table Function
make_combined_table <- function(data, treatment_var, covariates, digits = 2) {

  treatment_var <- rlang::ensym(treatment_var)

  # Pivot all covariates into long format

```

```

long_data <- data %>%
  pivot_longer(
    cols = all_of(covariates),
    names_to = "variable",
    values_to = "level"
  ) %>%
  mutate(level = as.character(level))

# Count by treatment + totals
tab <- long_data %>%
  count(variable, level, !!treatment_var) %>%
  pivot_wider(
    names_from = !!treatment_var,
    values_from = n,
    values_fill = 0
  )

# Rename treatment columns (assumes 0/1)
tab <- tab %>%
  rename(
    Untreated = `0`,
    Treated   = `1`
  )

# Add totals
tab <- tab %>%
  mutate(
    Total = Untreated + Treated
  )

# percentages
tab <- tab %>%
  group_by(variable) %>%
  mutate(
    Untreated = paste0(Untreated, " (", round(Untreated / sum(Untreated) * 100, digits), "%)"),
    Treated   = paste0(Treated, " (", round(Treated / sum(Treated) * 100, digits), "%)"),
    Total     = paste0(Total, " (", round(Total / sum(Total) * 100, digits), "%)"),
  ) %>%
  ungroup()

tab %>%
  arrange(variable, level)
}

table_combined <- make_combined_table(
  data = nodes,
  treatment_var = treatment,
  covariates = c(
    "posneg", "share", "education", "employment", "date"
  )
)

table_combined

```

```
## # A tibble: 10 x 5
##   variable level Untreated Treated Total
##   <chr>      <chr> <chr>      <chr>      <chr>
## 1 date      0      90 (47.87%) 14 (50%)    104 (48.15%)
## 2 date      1      98 (52.13%) 14 (50%)    112 (51.85%)
## 3 education 0      111 (59.04%) 15 (53.57%) 126 (58.33%)
## 4 education 1      77 (40.96%) 13 (46.43%) 90 (41.67%)
## 5 employment 0      105 (55.85%) 11 (39.29%) 116 (53.7%)
## 6 employment 1      83 (44.15%) 17 (60.71%) 100 (46.3%)
## 7 posneg    0      88 (46.81%) 14 (50%)    102 (47.22%)
## 8 posneg    1      100 (53.19%) 14 (50%)    114 (52.78%)
## 9 share     0      39 (20.74%) 11 (39.29%) 50 (23.15%)
## 10 share    1      149 (79.26%) 17 (60.71%) 166 (76.85%)
```

```
make_continuous_table <- function(data,
                                   treatment_var,
                                   cont_vars,
                                   digits = 2,
                                   show_sd = TRUE) {

  treatment_var <- rlang::ensym(treatment_var)

  map_dfr(cont_vars, function(var) {

    var_sym <- rlang::sym(var)

    # Means
    untreated_mean <- data %>%
      filter(!treatment_var == 0) %>%
      summarise(m = mean(!var_sym, na.rm = TRUE)) %>%
      pull(m)

    treated_mean <- data %>%
      filter(!treatment_var == 1) %>%
      summarise(m = mean(!var_sym, na.rm = TRUE)) %>%
      pull(m)

    overall_mean <- mean(data[[var]], na.rm = TRUE)

    if (show_sd) {
      untreated_sd <- data %>%
        filter(!treatment_var == 0) %>%
        summarise(s = sd(!var_sym, na.rm = TRUE)) %>%
        pull(s)

      treated_sd <- data %>%
        filter(!treatment_var == 1) %>%
        summarise(s = sd(!var_sym, na.rm = TRUE)) %>%
        pull(s)

      overall_sd <- sd(data[[var]], na.rm = TRUE)

      tibble(
        Variable = var,
```

```

    Untreated = sprintf("%.2f (%.2f)", untreated_mean, untreated_sd),
    Treated   = sprintf("%.2f (%.2f)", treated_mean, treated_sd),
    Total     = sprintf("%.2f (%.2f)", overall_mean, overall_sd)
  )

} else {

  tibble(
    Variable = var,
    Untreated = round(untreated_mean, digits),
    Treated   = round(treated_mean, digits),
    Total     = round(overall_mean, digits)
  )
}
})
}

table_continuous <- make_continuous_table(
  data = nodes,
  treatment_var = treatment,
  cont_vars = c("avg_posneg", "avg_share", "avg_education", "avg_employment", "avg_date")
)

table_continuous

```

```

## # A tibble: 5 x 4
##   Variable      Untreated   Treated     Total
##   <chr>         <chr>      <chr>      <chr>
## 1 avg_posneg    0.52 (0.33) 0.65 (0.37) 0.53 (0.33)
## 2 avg_share    0.77 (0.25) 0.77 (0.30) 0.77 (0.26)
## 3 avg_education 0.42 (0.32) 0.45 (0.37) 0.42 (0.33)
## 4 avg_employment 0.46 (0.30) 0.41 (0.35) 0.45 (0.31)
## 5 avg_date     0.51 (0.32) 0.67 (0.36) 0.53 (0.33)

```

Estimation using Inverse Probability Weighted Estimator (assumes exposure propensity score model correctly specified)

```

# -----
# 5. IPW Estimator
# -----

#Fit treatment model
mylogit <- glm(treatment ~ age10 + posneg + date + share + education + employment,
  data = nodes,
  family = "binomial")

#propensity score denominator
denom.p <- predict(mylogit,type = "response")

#stabilized weights
#numerator
numer.fit=glm(treatment~1,family=binomial(), data=nodes)
numer.p=predict(numer.fit,type="response")

```

```

#denominator

#unstabilized weights
nodes$w =ifelse(nodes$treatment==0,(1-denom.p),denom.p)

#stabilized weights
nodes$sw=ifelse(nodes$treatment==0,((1-numer.p)/(1-denom.p)),(numer.p/denom.p))

#check distribution of stabilized weights
#Mean around 1
#Deviation from 1 means possible model misspecification or positivity issue
summary(nodes$sw)

```

```

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.3206  0.9322  0.9740  1.0027  1.0340  2.8083

```

```

# IPW outcome model (marginal structural model) with robust variance
# Causal Risk Ratio
# Fit with stabilized weights
gee_rr_ipw <- geeglm(
  outcome ~ treatment,
  weights = sw,
  data = nodes,
  id = id,
  family = binomial(link = "log"),
  corstr = "independence"
)

```

```

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

```

```

#Warning in eval(family$initialize) :
#non-integer #successes in a binomial glm!
#Warning OK because we weighted geeglm

b_rr <- coef(gee_rr_ipw)["treatment"]
se_rr <- sqrt(vcov(gee_rr_ipw)["treatment", "treatment"])

rr <- exp(b_rr)
rr_ci <- exp(b_rr + c(-1, 1) * 1.96 * se_rr)

# Causal risk difference
# Fit with stabilized weights
gee_rd_ipw <- geeglm(
  outcome ~ treatment,
  weights = sw,
  data = nodes,
  id = id,
  family = gaussian(link = "identity"),
  corstr = "independence"
)

b_rd <- coef(gee_rd_ipw)["treatment"]
se_rd <- sqrt(vcov(gee_rd_ipw)["treatment", "treatment"])

rd <- b_rd

```

```
rd_ci <- b_rd + c(-1, 1) * 1.96 * se_rd

results_ipw <- data.frame(
  measure = c("Risk ratio", "Risk difference"),
  estimate = c(rr, rd),
  lower_95_ci = c(rr_ci[1], rd_ci[1]),
  upper_95_ci = c(rr_ci[2], rd_ci[2])
)

results_ipw$estimate <- round(results_ipw$estimate, 3)
results_ipw$lower_95_ci <- round(results_ipw$lower_95_ci, 3)
results_ipw$upper_95_ci <- round(results_ipw$upper_95_ci, 3)

results_ipw
```

```
##           measure estimate lower_95_ci upper_95_ci
## 1      Risk ratio    0.999      0.628      1.589
## 2 Risk difference   -0.001     -0.227      0.226
```

Estimation using the G-computation (outcome model)

```
# -----
# 6. G-computation
# -----

# Function for G-computation
# Save as a function so we can bootstrap to get confidence interval
# Under SUTVA and IID, we sample each individual with replacement for the bootstrap samples

gcomp_est <- function(data) {
  n <- nrow(data)

  # original data
  all <- data
  all$interv <- -1

  # counterfactual copy with A = 0
  all.untr <- all
  all.untr$interv <- 0
  all.untr$treatment <- 0
  all.untr$outcome <- NA

  # counterfactual copy with A = 1
  all.tr <- all
  all.tr$interv <- 1
  all.tr$treatment <- 1
  all.tr$outcome <- NA

  onesample <- as.data.frame(rbind(all, all.untr, all.tr))

  # outcome model
  # Use Binomial-Logit then take function of probabilities
  glm.obj <- glm(
```



```

    outcome ~ treatment + age10 + posneg + date + share + education + employment,
    data = onesample,
    family = binomial(link = "logit")
  )
summary(glm.obj)

onesample$meanY <- predict(glm.obj, newdata = onesample, type = "response")

m1 <- onesample$meanY[onesample$interv == 1]
m0 <- onesample$meanY[onesample$interv == 0]

risk1 <- mean(m1)
risk0 <- mean(m0)
rd <- risk1 - risk0
rr <- risk1 / risk0

c(risk1 = risk1, risk0 = risk0, rd = rd, rr = rr)
}

# Point estimates
est <- gcomp_est(nodes)
gcomp_rd <- est[3]
gcomp_rr <- est[4]

# Bootstrap CIs
set.seed(123)
B <- 1000 #Should be at least 1000 replicates for a bootstrap confidence interval

#Note: we bootstrap individuals here ignoring the network
#There are some instances that the model does not converge
boot_out <- replicate(B, {
  idx <- sample(seq_len(nrow(nodes)), replace = TRUE)
  gcomp_est(nodes[idx, ], drop = FALSE)
})

boot_out <- t(boot_out)

#Obtain 2.5% and 97.5% of bootstrap distribution for parameter of interest
ci <- apply(boot_out, 2, quantile, probs = c(0.025, 0.975), na.rm = TRUE)

gcomp_results <- data.frame(
  measure = c("Risk under A=1", "Risk under A=0", "Risk difference", "Risk ratio"),
  estimate = est[c("risk1", "risk0", "rd", "rr")],
  lower_95_ci = ci[1, c("risk1", "risk0", "rd", "rr")],
  upper_95_ci = ci[2, c("risk1", "risk0", "rd", "rr")]
)

gcomp_results$estimate <- round(gcomp_results$estimate, 3)
gcomp_results$lower_95_ci <- round(gcomp_results$lower_95_ci, 3)
gcomp_results$upper_95_ci <- round(gcomp_results$upper_95_ci, 3)

gcomp_results

##           measure estimate lower_95_ci upper_95_ci

```

## risk1	Risk under A=1	0.487	0.319	0.658
## risk0	Risk under A=0	0.486	0.417	0.556
## rd	Risk difference	0.002	-0.179	0.184
## rr	Risk ratio	1.003	0.649	1.422

```
#save confidence intervals
```

```
gcomp_rd_ci <- c(gcomp_results$lower_95_ci[3],gcomp_results$upper_95_ci[3])
```

```
gcomp_rr_ci <- c(gcomp_results$lower_95_ci[4],gcomp_results$upper_95_ci[4])
```

Estimation using an Augmented IPW Estimator (doubly robust). Two estimators coded below are augmented IPW (DR1, Lunceford and Davidian 2004) and weighted regression estimator (DR2, Robins 2007).

```
#####
```

```
# 7. Doubly Robust Estimators
```

```
#####
```

```
# Outcome model for g-formula components used in DR1
```

```
nodes_obs <- nodes
```

```
nodes_obs$interv <- -1
```

```
nodes_0 <- nodes
```

```
nodes_0$interv <- 0
```

```
nodes_0$treatment <- 0
```

```
nodes_0$outcome <- NA
```

```
nodes_1 <- nodes
```

```
nodes_1$interv <- 1
```

```
nodes_1$treatment <- 1
```

```
nodes_1$outcome <- NA
```

```
stacked <- rbind(nodes_obs, nodes_0, nodes_1)
```

```
outcome_mod <- glm(
```

```
  outcome ~ treatment + age10 + posneg + date + share + education + employment,
```

```
  data = stacked,
```

```
  family = binomial(link = "logit")
```

```
)
```

```
stacked$mean_outcome <- predict(outcome_mod, newdata = stacked, type = "response")
```

```
nodes$m0 <- stacked$mean_outcome[stacked$interv == 0]
```

```
nodes$m1 <- stacked$mean_outcome[stacked$interv == 1]
```

```
nodes$denom.p <- denom.p
```

```
# DR1: augmented IPW / Lunceford-Davidian
```

```
dr1_risk1 <- mean(
```

```
(
```

```
  nodes$treatment * nodes$outcome - (nodes$treatment - nodes$denom.p) * nodes$m1) / nodes$denom.p
```

```
)
```

```
dr1_risk0 <- mean(
```

```
(
```

```
  (1 - nodes$treatment) * nodes$outcome + (nodes$treatment - nodes$denom.p) * nodes$m0) / (1 - nodes$denom.p)
```

```

)

dr1_rd <- dr1_risk1 - dr1_risk0
dr1_rr <- dr1_risk1 / dr1_risk0

# DR2: weighted regression / Robins
nodes$ipw <- with(nodes, treatment / denom.p + (1 - treatment) / (1 - denom.p))

dr2_rd_mod <- lm(
  outcome ~ treatment + age10 + posneg + date + share + education + employment,
  data = nodes,
  weights = ipw
)

dr2_rr_mod <- glm(
  outcome ~ treatment + age10 + posneg + date + share + education + employment,
  data = nodes,
  weights = ipw,
  family = binomial(link = "logit")
)

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
#Warning in eval(family$initialize) :
#non-integer #successes in a binomial glm!
#Warning OK because we weighted glm

# Causal risk difference

nodes_1_dr <- nodes
nodes_1_dr$treatment <- 1

nodes_0_dr <- nodes
nodes_0_dr$treatment <- 0

dr2_rd_m1 <- predict(dr2_rd_mod, newdata = nodes_1_dr, type = "response")
dr2_rd_m0 <- predict(dr2_rd_mod, newdata = nodes_0_dr, type = "response")
dr2_rd <- mean(dr2_rd_m1) - mean(dr2_rd_m0)

dr2_rr_m1 <- predict(dr2_rr_mod, newdata = nodes_1_dr, type = "response")
dr2_rr_m0 <- predict(dr2_rr_mod, newdata = nodes_0_dr, type = "response")
dr2_risk1 <- mean(dr2_rr_m1)
dr2_risk0 <- mean(dr2_rr_m0)
dr2_rr <- dr2_risk1 / dr2_risk0

dr_results <- data.frame(
  estimator = c("DR1", "DR1", "DR2", "DR2"),
  measure = c("Risk difference", "Risk ratio", "Risk difference", "Risk ratio"),
  estimate = c(dr1_rd, dr1_rr, dr2_rd, dr2_rr)
)

dr_results

## estimator      measure      estimate
## 1      DR1 Risk difference 0.0006497218

```

```

## 2      DR1      Risk ratio 1.0013315917
## 3      DR2 Risk difference 0.0083692436
## 4      DR2      Risk ratio 1.0294598410

#save results

dr1_rd <- dr_results$estimate[1]
dr1_rr <- dr_results$estimate[2]
dr2_rd <- dr_results$estimate[3]
dr2_rr <- dr_results$estimate[4]

#####
# 8. Doubly Robust Estimators - Bootstrap CIs
#####
# Under SUTVA and IID, we sample each individual with replacement for the boot samples
#Turn into a function for DR1 and DR2 to bootstrap to get 95% CIs

estimate_effects <- function(data) {
  # -----
  # G-computation
  # -----
  dat_obs <- data
  dat_obs$interv <- -1

  dat_0 <- data
  dat_0$interv <- 0
  dat_0$treatment <- 0
  dat_0$outcome <- NA

  dat_1 <- data
  dat_1$interv <- 1
  dat_1$treatment <- 1
  dat_1$outcome <- NA

  stacked <- rbind(dat_obs, dat_0, dat_1)

  withCallingHandlers(
    {
      outcome_mod <- glm(
        outcome ~ treatment + age10 + date + posneg + share + education + employment,
        data = stacked,
        family = binomial(link = "logit")
      )
    },
    #suppress warning for weighting in the glm
    #This warning is expected because we have binary outcomes
    warning = function(w) {
      if (grepl("non-integer #successes in a binomial glm!", conditionMessage(w))) {
        invokeRestart("muffleWarning")
      }
    }
  )

  stacked$mean_outcome <- predict(outcome_mod, newdata = stacked, type = "response")

```

```

m1 <- stacked$mean_outcome[stacked$interv == 1]
m0 <- stacked$mean_outcome[stacked$interv == 0]

g_risk1 <- mean(m1)
g_risk0 <- mean(m0)
g_rd <- g_risk1 - g_risk0
g_rr <- g_risk1 / g_risk0

data$m1 <- m1
data$m0 <- m0

# -----
# DR1: augmented IPW
# w is the propensity score P(treatment = 1 | L)
# -----

dr1_risk1 <- mean(
(
data$treatment * data$outcome - (data$treatment - data$denom.p) * data$m1) / data$denom.p
)

dr1_risk0 <- mean(
(
(1 - data$treatment) * data$outcome + (data$treatment - data$denom.p) * data$m0) / (1 - data$denom.p)
)

dr1_rd <- dr1_risk1 - dr1_risk0
dr1_rr <- dr1_risk1 / dr1_risk0

# -----
# DR2: weighted regression
# -----

ipw <- data$treatment / data$denom.p + (1 - data$treatment) / (1 - data$denom.p)
data$ipw <- ipw

wreg_rd <- lm(
outcome ~ treatment + age10 + date + posneg + share + education + employment,
data = data,
weights = ipw
)

withCallingHandlers(
{
wreg_rr <- glm(
outcome ~ treatment + age10 + date + posneg + share + education + employment,
data = data,
weights = ipw,
family = binomial(link = "logit")
)
},
#suppress warning for weighting in the glm
#This warning is expected because we have binary outcomes

```

```

warning = function(w) {
  if (grepl("non-integer #successes in a binomial glm!", conditionMessage(w))) {
    invokeRestart("muffleWarning")
  }
}
)

dat1 <- data
dat1$treatment <- 1

dat0 <- data
dat0$treatment <- 0

rd_m1 <- predict(wreg_rd, newdata = dat1, type = "response")
rd_m0 <- predict(wreg_rd, newdata = dat0, type = "response")
dr2_rd <- mean(rd_m1) - mean(rd_m0)

rr_m1 <- predict(wreg_rr, newdata = dat1, type = "response")
rr_m0 <- predict(wreg_rr, newdata = dat0, type = "response")
dr2_risk1 <- mean(rr_m1)
dr2_risk0 <- mean(rr_m0)
dr2_rr <- dr2_risk1 / dr2_risk0

c(
  gcomp_rd = g_rd,
  gcomp_rr = g_rr,
  dr1_rd = dr1_rd,
  dr1_rr = dr1_rr,
  dr2_rd = dr2_rd,
  dr2_rr = dr2_rr
)
}

set.seed(123)
B <- 1000

point_est <- estimate_effects(nodes)

boot_est <- replicate(B, {
  idx <- sample(seq_len(nrow(nodes)), replace = TRUE)
  estimate_effects(nodes[idx, , drop = FALSE])
})

boot_est <- t(boot_est)

ci <- apply(boot_est, 2, quantile, probs = c(0.025, 0.975), na.rm = TRUE)

results_dr <- data.frame(
  estimator = c("G-computation", "G-computation", "DR1", "DR1", "DR2", "DR2"),
  measure = c("Risk difference", "Risk ratio",
              "Risk difference", "Risk ratio",
              "Risk difference", "Risk ratio"),

```

```

estimate = point_est[c("gcomp_rd", "gcomp_rr", "dr1_rd", "dr1_rr", "dr2_rd", "dr2_rr")],
lower_95_ci = ci[1, c("gcomp_rd", "gcomp_rr", "dr1_rd", "dr1_rr", "dr2_rd", "dr2_rr")],
upper_95_ci = ci[2, c("gcomp_rd", "gcomp_rr", "dr1_rd", "dr1_rr", "dr2_rd", "dr2_rr")]
)

results_dr$estimate <- round(results_dr$estimate, 3)
results_dr$lower_95_ci <- round(results_dr$lower_95_ci, 3)
results_dr$upper_95_ci <- round(results_dr$upper_95_ci, 3)

results_dr

##           estimator      measure estimate lower_95_ci upper_95_ci
## gcomp_rd G-computation Risk difference    0.002    -0.179    0.184
## gcomp_rr G-computation      Risk ratio    1.003     0.649    1.422
## dr1_rd      DR1 Risk difference    0.001    -0.189    0.187
## dr1_rr      DR1      Risk ratio    1.001     0.634    1.424
## dr2_rd      DR2 Risk difference    0.008    -0.170    0.186
## dr2_rr      DR2      Risk ratio    1.029     0.670    1.418

dr1_rd_ci <- c(results_dr$lower_95_ci[3], results_dr$upper_95_ci[3])
dr1_rr_ci <- c(results_dr$lower_95_ci[4], results_dr$upper_95_ci[4])
dr2_rd_ci <- c(results_dr$lower_95_ci[5], results_dr$upper_95_ci[5])
dr2_rr_ci <- c(results_dr$lower_95_ci[6], results_dr$upper_95_ci[6])

# Suppressed: Warning in eval(family$initialize): non-integer #successes in a binomial glm!

#Code to output results to Latex and Forest Plot (optional)

z <- qnorm(0.975)

make_rr_row <- function(estimator, fit, term = "treatment") {
  b <- coef(fit)[term]
  se <- sqrt(vcov(fit)[term, term])
  est <- exp(b)
  ci <- exp(b + c(-1, 1) * z * se)

  data.frame(
    Estimator = estimator,
    Measure = "Risk ratio",
    Estimate = est,
    LCL = ci[1],
    UCL = ci[2]
  )
}

make_rd_row <- function(estimator, fit, term = "treatment") {
  b <- coef(fit)[term]
  se <- sqrt(vcov(fit)[term, term])
  ci <- b + c(-1, 1) * z * se

  data.frame(
    Estimator = estimator,
    Measure = "Risk difference",
    Estimate = b,

```

```

    LCL = ci[1],
    UCL = ci[2]
  )
}

make_manual_row <- function(estimator, measure, est, ci) {
  data.frame(
    Estimator = estimator,
    Measure = measure,
    Estimate = est,
    LCL = ci[1],
    UCL = ci[2]
  )
}

results <- bind_rows(
  make_rr_row("Unadjusted", model_rr_unadj),
  make_rd_row("Unadjusted", model_rd_unadj),

  make_rr_row("Covariate adjusted", gee_rr),
  make_rd_row("Covariate adjusted", gee_rd),

  make_rr_row("IPW", gee_rr_ipw),
  make_rd_row("IPW", gee_rd_ipw),

  make_manual_row("G-computation", "Risk ratio", gcomp_rr, gcomp_rr_ci),
  make_manual_row("G-computation", "Risk difference", gcomp_rd, gcomp_rd_ci),

  make_manual_row("DR1", "Risk ratio", dr1_rr, dr1_rr_ci),
  make_manual_row("DR1", "Risk difference", dr1_rd, dr1_rd_ci),

  make_manual_row("DR2", "Risk ratio", dr2_rr, dr2_rr_ci),
  make_manual_row("DR2", "Risk difference", dr2_rd, dr2_rd_ci)
)

table_wide <- results %>%
  mutate(
    Estimate = round(Estimate, 3),
    LCL = round(LCL, 3),
    UCL = round(UCL, 3),
    value = sprintf("%.3f (%.3f, %.3f)", Estimate, LCL, UCL)
  ) %>%
  select(Estimator, Measure, value) %>%
  pivot_wider(names_from = Measure, values_from = value) %>%
  select(Estimator, `Risk ratio`, `Risk difference`)

latex_tab <- kable(
  table_wide,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  align = c("l", "l", "l")
)

```



```

)

slide_tex <- paste0(
  "\\begin{frame}{Effect Estimates}\\n",
  "\\scriptsize\\n",
  "\\centering\\n",
  latex_tab,
  "\\n\\end{frame}\\n"
)

writeLines(slide_tex, "effect_estimates_slide.tex")

# Keep only causal estimators for the forest plot

forest_data <- results %>%
  filter(
    Estimator %in% c(
      "Unadjusted",
      "Covariate adjusted",
      "IPW",
      "G-computation",
      "DR1",
      "DR2"
    )
  ) %>%
  mutate(
    Estimator = factor(
      Estimator,
      levels = rev(c(
        "Unadjusted",
        "Covariate adjusted",
        "IPW",
        "G-computation",
        "DR1",
        "DR2"
      ))
    )
  )

# Colors matched to the slide
uri_teal <- "#349B9B"
uri_navy <- "#0B2E63"
uri_gray <- "#B8B8B8"
uri_light_gray <- "#F2F4F5"

estimator_colors <- c(
  "Unadjusted" = uri_navy,
  "Covariate adjusted" = uri_teal,
  "IPW" = "#5CB8B2",
  "G-computation" = "#2F8F8F",
  "DR1" = "#0B2E63",
  "DR2" = "#6AA6A6"
)

```

```

forest_rr <- forest_data %>%
  filter(Measure == "Risk ratio") %>%
  ggplot(aes(x = Estimate, y = Estimator, color = Estimator)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = uri_navy, linewidth = 0.5) +
  geom_errorbarh(aes(xmin = LCL, xmax = UCL), height = 0.15, linewidth = 0.8) +
  geom_point(size = 2.2) +
  scale_color_manual(values = estimator_colors, drop = FALSE) +
  labs(
    title = "Risk Ratio",
    x = "Estimate and 95% CI",
    y = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(color = uri_navy, face = "bold"),
    axis.text.y = element_text(color = uri_navy, size = 14),
    axis.title.x = element_text(color = uri_navy),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),
    panel.grid.major.x = element_line(color = uri_light_gray),
    legend.position = "none"
  )

```

```

## Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
## i Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

```

forest_rd <- forest_data %>%
  filter(Measure == "Risk difference") %>%
  ggplot(aes(x = Estimate, y = Estimator, color = Estimator)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = uri_navy, linewidth = 0.5) +
  geom_errorbarh(aes(xmin = LCL, xmax = UCL), height = 0.15, linewidth = 0.8) +
  geom_point(size = 2.2) +
  scale_color_manual(values = estimator_colors, drop = FALSE) +
  labs(
    title = "Risk Difference",
    x = "Estimate and 95% CI",
    y = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(color = uri_navy, face = "bold"),
    axis.text.y = element_text(color = uri_navy, size = 14),
    axis.title.x = element_text(color = uri_navy),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),
    panel.grid.major.x = element_line(color = uri_light_gray),
    legend.position = "none"
  )

ggsave(
  filename = "effect_estimates_forest_plot_rr.pdf",

```

```

plot = forest_rr,
width = 8,
height = 7
)

```

`height` was translated to `width`.

```

ggsave(
  filename = "effect_estimates_forest_plot_rd.pdf",
  plot = forest_rd,
  width = 8,
  height = 7
)

```

`height` was translated to `width`.

Assess the fit of the propensity score model using overlap plots and standardized mean differences before and after weighting.

```

#####
# 9. Assess the propensity score model and outcome model fit
#####

# IPW from propensity score
nodes$ipw <- with(nodes, treatment / denom.p + (1 - treatment) / (1 - denom.p))

# Density plot of propensity scores by treatment group
overlap <- ggplot(nodes, aes(x = denom.p, color = factor(treatment), fill = factor(treatment))) +
  geom_density(alpha = 0.25) +
  labs(
    x = "Propensity score",
    y = "Density",
    color = "Treatment",
    fill = "Treatment",
    title = "Propensity score overlap by treatment group"
  ) +
  theme_minimal()

ggsave(
  filename = "propensity_overlap.pdf",
  plot = overlap,
  width = 8,
  height = 7
)

#Standardized Mean Differences
bal <- bal.tab(
  treatment ~ age10 + date + posneg + share + education + employment,
  data = nodes,
  weights = nodes$ipw,
  un = TRUE,
  method = "weighting",
  estimand = "ATE",
  #binary="std",
  continuous="std"
)

```

```
print(bal)
```

```
## Balance Measures
##           Type Diff.Un Diff.Adj
## age10     Contin. -0.3119 -0.0014
## date      Binary -0.0213 -0.0330
## posneg     Binary -0.0319  0.0855
## share      Binary -0.1854  0.0214
## education  Binary  0.0547  0.0338
## employment Binary  0.1657 -0.0381
##
## Effective sample sizes
##           Control Treated
## Unadjusted  188.      28.
## Adjusted    186.75   20.64
```

```
smd_table <- bal$Balance
smd_table[, c("Diff.Un", "Diff.Adj")]
```

```
##           Diff.Un    Diff.Adj
## age10     -0.31192909 -0.001435053
## date      -0.02127660 -0.032982923
## posneg     -0.03191489  0.085505547
## share      -0.18541033  0.021381623
## education  0.05471125  0.033832176
## employment 0.16565350 -0.038061197
```

```
#Love Plot
```

```
love <- love.plot(
  bal,
  stat = "mean.diffs",
  abs = TRUE,
  thresholds = c(m = 0.1),
  var.order = "unadjusted",
  title = "Absolute standardized mean differences before and after IPW"
)
```

```
## Warning: Standardized mean differences and raw mean differences are present in the same
## plot. Use the `stars` argument to distinguish between them and appropriately
## label the x-axis. See `love.plot()` for details.
```

```
ggsave(
  filename = "loveplot.pdf",
  plot = love,
  width = 8,
  height = 7
)
```

```
# Check fit of GEE model
```

```
glm.obj <- glm( outcome ~ treatment + age10 + posneg
  + date + share + education + employment, data = nodes,
  family = binomial(link = "logit") )
```

```

nodes$pred <- predict(glm.obj, type = "response")

nodes$pred_group <- cut( nodes$pred, breaks = quantile(nodes$pred, seq(0, 1, 0.1), na.rm = TRUE), include
cal <- aggregate( cbind(observed = nodes$outcome, predicted = nodes$pred), by = list(pred_group = nodes

outcomefit <- ggplot(
  cal,
  aes(x = predicted, y = observed)
) +
  geom_point(size = 3) +
  geom_abline(
    intercept = 0,
    slope = 1,
    linetype = "dashed"
  ) +
  coord_equal(
    xlim = c(0, 1),
    ylim = c(0, 1)
  ) +
  labs(
    x = "Mean predicted outcome",
    y = "Mean observed outcome"
  ) +
  theme_classic(base_size = 14)

ggsave(
  filename = "outcomefit.pdf",
  plot = outcomefit,
  width = 8,
  height = 7
)

```